DM1 treatment-response analysis: full report

ACoP 2026 Hackathon, Team 1. Written for Euibeom Shin, 11 October 2026. Every number here comes from `analysis/dm1_analysis.R` run on the real dataset, or from the exploration scripts in `analysis/explorations/euibeom/`. Figures are in `analysis/dm1_outputs/`.

1. The question and the answer

Question set by the organizers. In each of three treatment arms, which baseline characteristics predict who responds, and can machine learning find more than ordinary covariate models do? Some of the 18 covariates are noise on purpose.

Answer. One covariate dominates each arm, every method agrees on it, and it fits the drug's mechanism. Machine learning finds the same covariates. It predicts better only in the Tideglusib arm, and only because the effect there is a step rather than a slope; a linear model given that step catches up.

Arm	Outcome (good direction)	Key covariate	Clinical reading	Prediction on held-out patients
Mexiletine	Change in myotonia at 12 weeks (lower)	`myotonia_severity`, with `ctg_repeats` as a near-twin	A drug that quiets over-excited muscle helps most where myotonia is worst	R^2 about 0.5, simple models tie with ML
Oligonucleotide	Change in muscle-strength score at 26 weeks (higher)	`ctg_repeats`	A drug that lowers the toxic RNA helps most where there is most of it	R^2 under 0.1 for every method; responder AUC about 0.72
Tideglusib	Combined cognitive and motor score, 0 to 20 (higher)	`cognitive_function`	A multisystem drug helps most where cognition is already affected	R^2 0.56 for random forest, 0.45 for a linear model with a cut-off, 0.24 to 0.30 for straight lines

The noise probe, a column of random numbers added as a control, was never selected in any arm.

2. The data

500 patients, one row each, 26 columns. Each patient received one drug, so the file is really three small studies side by side.

Arm	Patients	Outcome column	Responder label	Responders
Mexiletine	161	`myotonia_change_12wk`	Fell by 45% or more of baseline myotonia	29%
Oligonucleotide	170	`qmt_change_26wk`	Change of about 1.37 or more	41%
Tideglusib	169	`tideglusib_composite`	Score of 6 or more	48%

The 18 baseline covariates, grouped by what they measure:

Group	Covariates
Genetics	`ctg_repeats`, `somatic_mosaicism_index`
Disease course	`phenotype` (onset category), `age`, `age_at_onset`, `disease_duration`, `family_history`
Muscle	`myotonia_severity`, `muscle_strength`, `ck_level`
Other organs	`cognitive_function` (a z-score, 0 = average), `cataracts`, `pr_interval` (heart conduction), `insulin_resistance`
General	`sex`, `bmi`, `albumin`, `smoking_status`

What this means for the analysis. Three separate analyses, one per arm, each with about 165 patients. No placebo group, so we can say who does well on a drug but not that the drug caused it. 18 covariates for 165 patients is a lot, so most of the later steps exist to stop a covariate being picked by chance.

3. Cleaning

Checks are done on baseline columns and plausibility only, never by looking at the outcome. The code is the `note()` block in `R/common.R`; it prints every finding and writes them to `data_checks.txt`.

Fine: no missing values, no duplicate patients, no spelling variants, age minus age at onset equals disease duration for everyone, every responder label matches its outcome.

Finding	Likely explanation	Decision
`albumin` is exactly 4.18 for all 161 Mexiletine patients	4.18 is the median of the whole dataset. The column was probably not measured in that arm and filled in. This is the organizers' "no missing data, but needs cleaning"	Dropped as a predictor in the Mexiletine arm only. Automatic: any covariate with a single value among an arm's training patients is dropped there
`ck_level` is exactly 50 for 32% of patients, none lower	A floor, such as a lab's lowest reportable value	Keep as is; state it on the slide
`somatic_mosaicism_index` is exactly 0 for 15%	Real: these patients have short repeats (median 107 against 316), and short repeats are stable	Keep as is
9 `albumin` values flagged as extreme (3.14 to 5.35)	Flagged only because most values sit close to 4.2; all nine are medically plausible	Keep as is

No patient is removed.

4. The split, and cross-validation

One split, done once: training versus test

Before any modelling, a quarter of each arm's patients is set aside as the test set and not scored until the end. The test set stands in for new patients; scoring on it once gives an honest number. If test patients influence any earlier choice, that is leakage, which the organizers called the most common mistake.

- Per arm, 75% training and 25% test.
- Balanced: the same share of responders and the same onset mix on both sides. The organizers' example of a bad split was juvenile-onset patients landing only in the test set.
- Fixed seed (`20261011`), so every teammate gets identical sides.

Arm	Training	Test	Responders, train / test	Adult / juvenile / late onset, train	Same, test
Mexiletine	120	41	29% / 29%	59 / 23 / 18%	59 / 24 / 17%
Oligonucleotide	127	43	41% / 42%	61 / 18 / 20%	60 / 19 / 21%
Tideglusib	128	41	48% / 49%	65 / 19 / 16%	66 / 17 / 17%

Five folds, inside the training set: cross-validation

Many choices must be made before the final exam: the LASSO penalty, the XGBoost settings, the cut-off in the Tideglusib model, whether an exploration idea helps. These are judged by five-fold cross-validation. The training patients are divided into five groups of about 25; five times, a model learns on four groups and is checked on the fifth, so every training patient is checked once by a model that never saw them. The folds keep the responder share and use the fixed seed (`cv_folds()`).

> **Rule for the slides.** Cross-validation is how we chose; the test set is how we report. The two kinds of score never sit in the same table.

Honesty note: the test set has been scored twice, once in the first run and once after the changes in section 11. The changes were decided on training data only, and both runs are logged in `DECISIONS.md`.

5. How the covariates relate to each other

Before selecting anything, check which covariates move together, because when two carry the same information a method picks one of them more or less by chance. Spearman correlation on training patients, similar in all three arms:

Pair	r	Meaning
`myotonia_severity` ~ `ctg_repeats`	0.88 to 0.90	Nearly one signal: longer repeat, worse myotonia
`age_at_onset` ~ `age`	0.84 to 0.89	Mostly one signal
`muscle_strength` ~ `disease_duration`	-0.63 to -0.78	Longer disease, weaker muscles
`somatic_mosaicism_index` ~ `ctg_repeats`	0.55 to 0.73	Longer repeats expand more over a lifetime
`age_at_onset` ~ `ctg_repeats`	-0.52 to -0.61	Longer repeat, earlier onset

`phenotype` explains about 80% of `age_at_onset` and 70% of `age`. Median repeat length: juvenile onset about 450 to 520, adult about 300, late about 100.

So the 18 covariates form two clusters plus loose extras. **Severity cluster:** `ctg_repeats`, `myotonia_severity`, `somatic_mosaicism_index`, `phenotype`, `age_at_onset`. **Disease-course cluster:** `disease_duration`, `muscle_strength`, `age`. **Loose:** everything else, including `cognitive_function`. This is textbook DM1 biology, which fits the organizers' statement that the data was built from current knowledge. Two consequences: a selected covariate from a cluster may be standing in for the cluster, and `cognitive_function`, which correlates with nothing much, stands alone when selected.

6. Selecting covariates with the simple models

Three ways of asking a linear model which covariates it needs, all on training patients:

Method	How it chooses	Weakness
Stepwise (BIC)	Adds or drops one covariate at a time; keeps a change only if the fit improves by more than a penalty for complexity	Unstable with correlated covariates: the order decides which of a pair survives
LASSO	Fits all covariates at once but shrinks every coefficient toward zero; weak ones reach exactly zero and drop out. Shrinkage strength chosen by cross-validation	Keeps one of a correlated pair and zeroes the other, somewhat arbitrarily
Elastic net	LASSO plus a second penalty that lets correlated covariates share the credit	Keeps a few more covariates than LASSO

Two guards against picking by chance. (1) A **noise probe**, a column of random numbers added before fitting; anything scoring at or below it is noise to that method. (2) **Stability**: redraw the training set 200 times with replacement, run LASSO each time, and count how often each covariate is kept. A real signal survives most redraws.

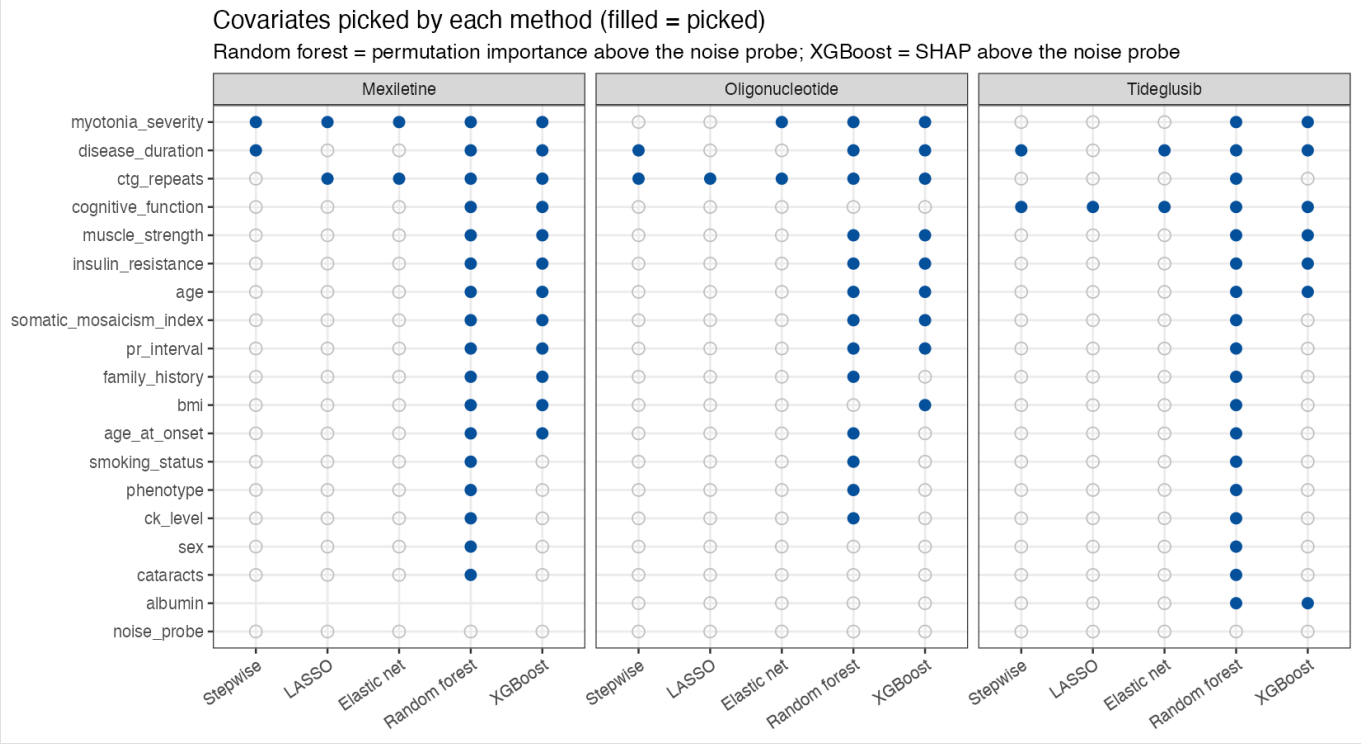


Figure 1. Which method picked which covariate. Filled dot = picked. Read down the first three columns of each panel: the simple models agree on one covariate per arm (two in Mexiletine). The random-forest and XGBoost columns are fuller because their bar, "above the noise probe", is low; that is why the final call also requires stability (section 8).

7. Machine learning, and reading importance out of it

Model	How it works	Why include it
Random forest	500 decision trees, each grown on a random subset of patients and covariates; answers are averaged	Handles thresholds and interactions on its own; hard to overfit with default settings
XGBoost	Small trees grown one after another, each correcting the errors of the last	Often the strongest model on tabular clinical data, but overfits quickly on a small arm

XGBoost is fitted twice: untuned (defaults, 100 rounds) and tuned, where tree depth (1, 2 or 3) and learning rate (0.03 or 0.1) are chosen by cross-validation, with early stopping on the number of rounds. The grid is deliberately small because each arm has only about 125 training patients. In every arm the tuned model chose depth 1, that is, stumps: a sign that there are no strong interactions to find.

Tree models give no coefficients, so importance is read two ways. **Permutation importance** (random forest): shuffle one covariate's values across patients and measure how much worse the model gets. **SHAP** (XGBoost): split each patient's prediction into a contribution per covariate, like an itemised receipt; averaging the absolute contributions ranks covariates, and the sign shows direction.

Arm	Leader (permutation importance)	Runners-up	Noise probe
Mexiletine	`myotonia_severity` 1.00	`ctg_repeats` 0.45, `age` 0.12	below zero, last of 18
Oligonucleotide	`ctg_repeats` 0.44	`myotonia_severity` 0.26, `muscle_strength` 0.14	about zero, 14th of 19
Tideglusib	`cognitive_function` 5.48	`muscle_strength` 0.88, `disease_duration` 0.33	below zero, last of 19

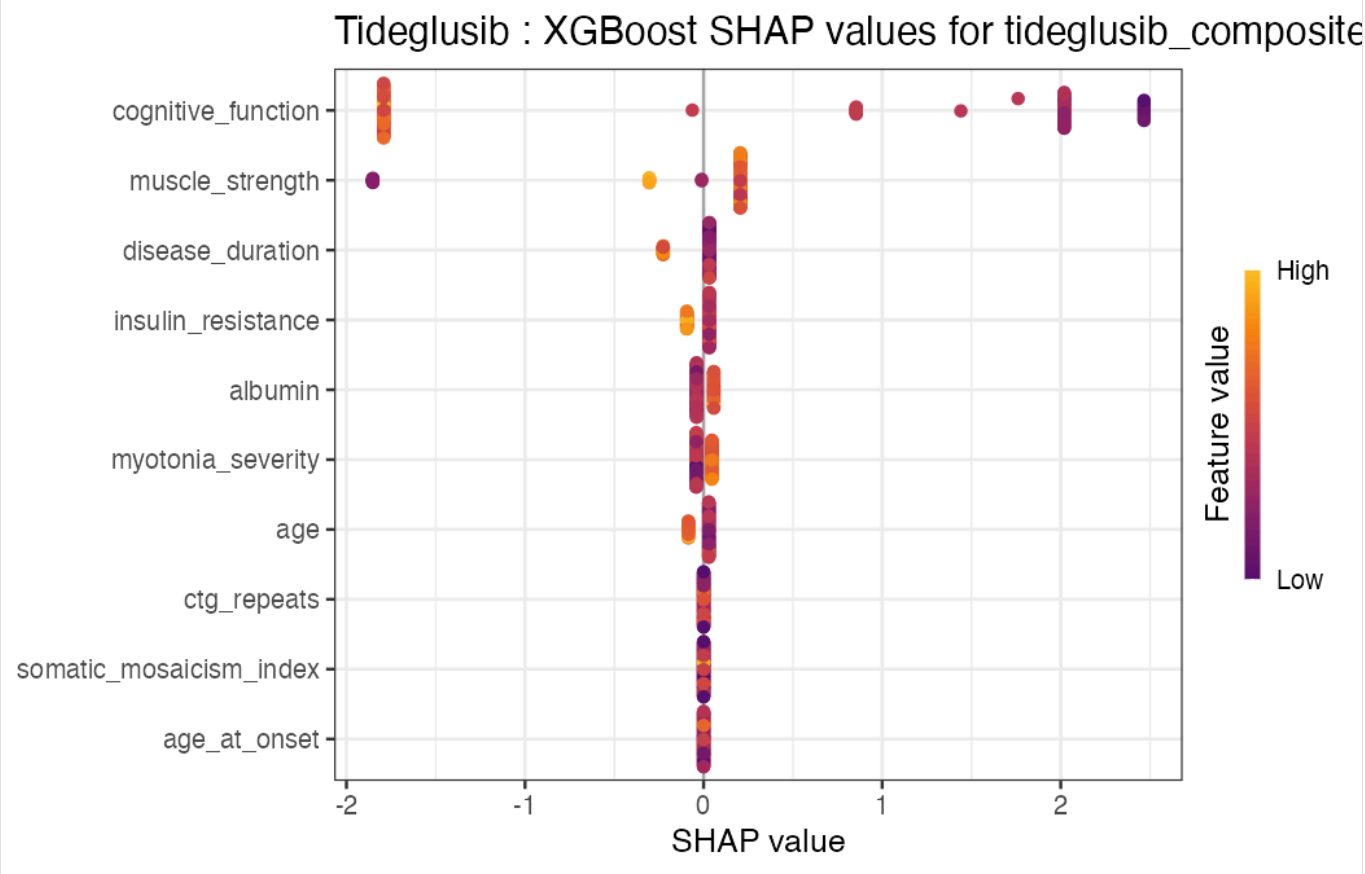


Figure 2. Tideglusib, SHAP values from the tuned XGBoost model (training patients). Each dot is one patient; its position is how much that covariate pushed the patient's predicted score up or down; colour is the patient's covariate value. `cognitive_function` forms two clumps: low values (dark) push the score up by about +2, high values (yellow) push it down by about -1.8, with almost nothing in between. That is the step described in section 9. Every other covariate hugs zero.

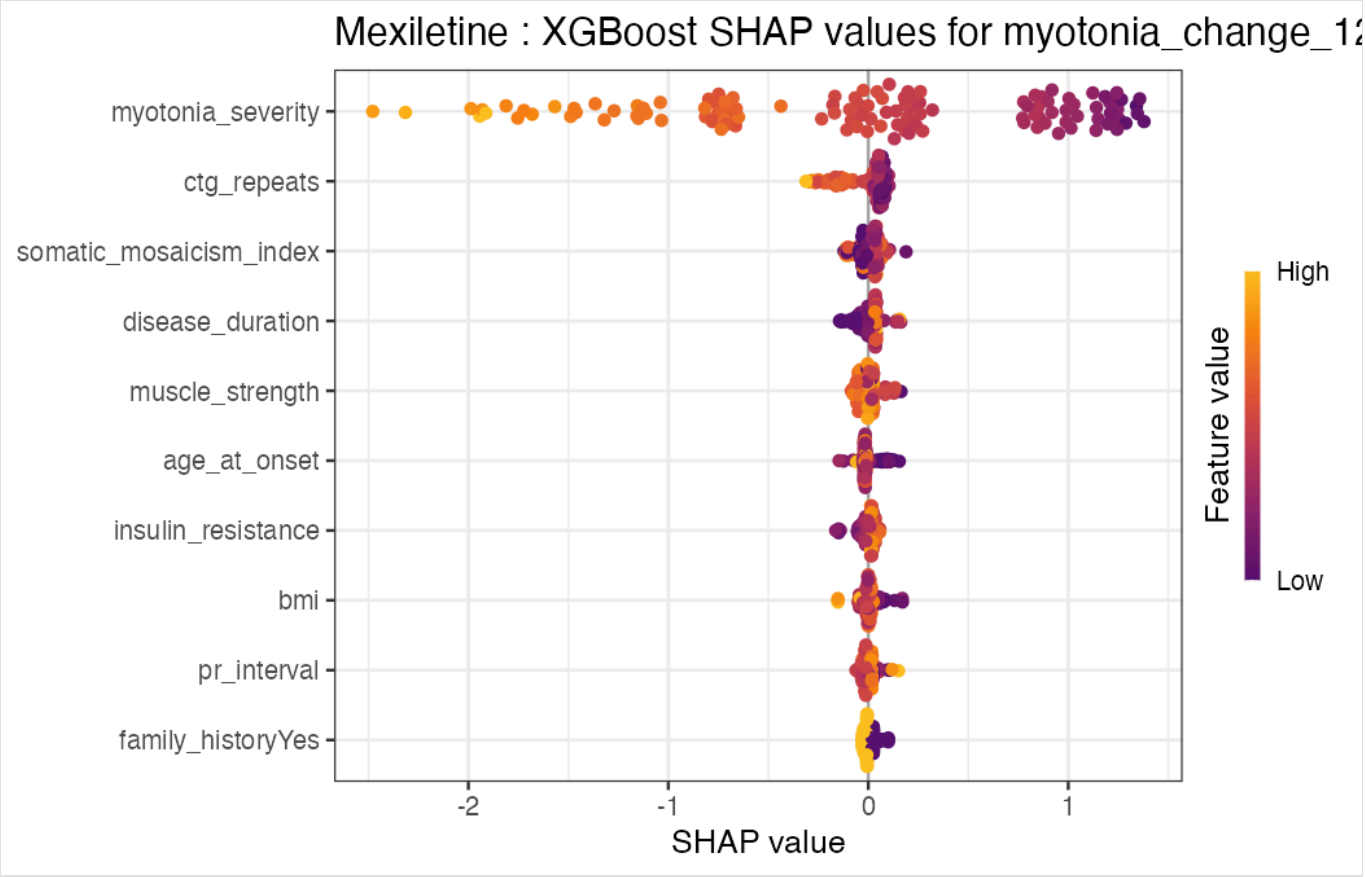


Figure 3. Mexiletine, SHAP values. High `myotonia_severity` (yellow) pushes the predicted change strongly negative, which is the good direction in this arm. `ctg_repeats` adds a smaller push the same way; the rest hug zero.

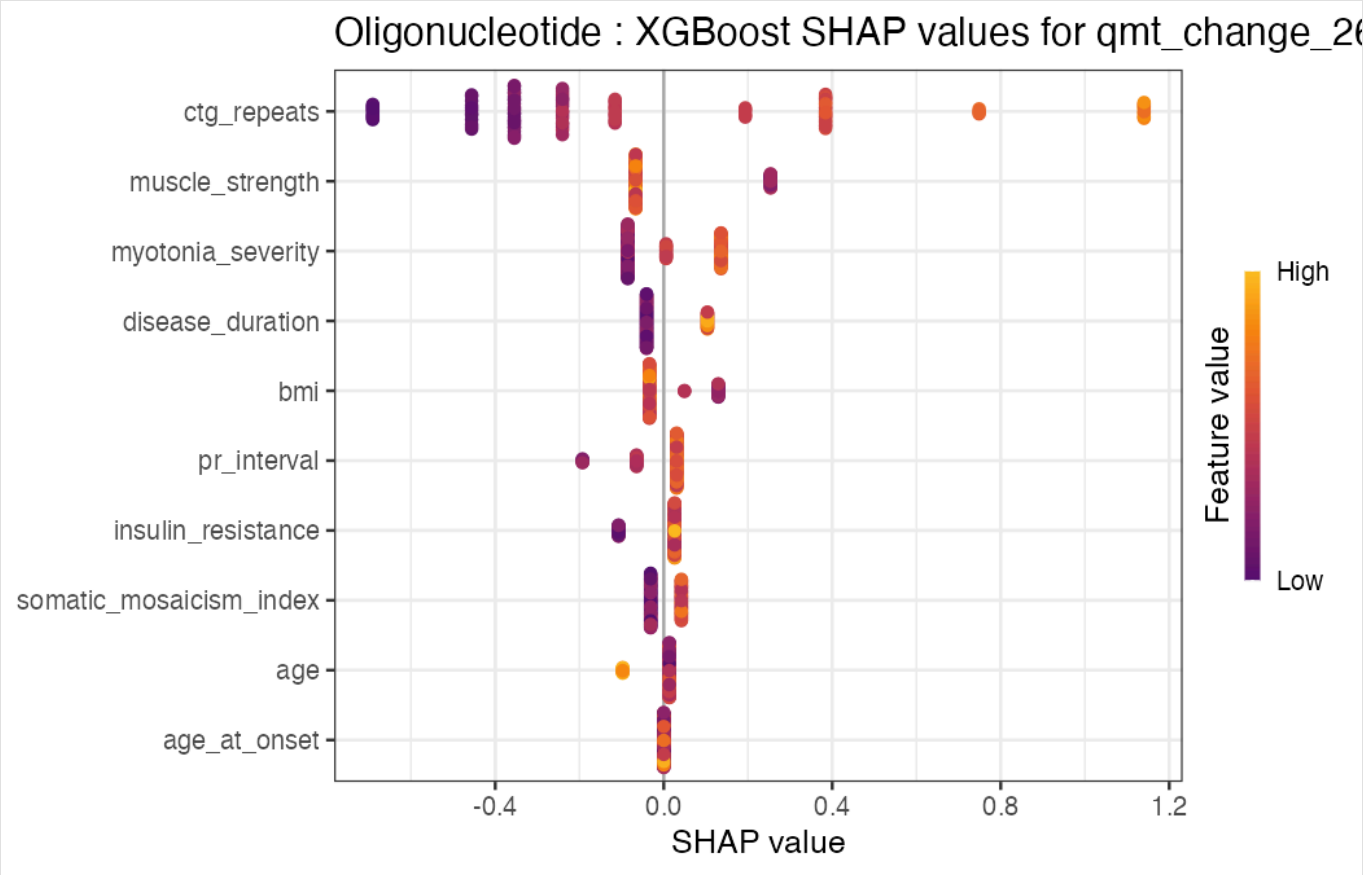


Figure 4. Oligonucleotide, SHAP values. `ctg_repeats` runs from about -0.7 (short repeats) to +1.1 (long repeats), a steady slope. Note the small axis range compared with the other arms: the model's total pull is weak here.

What the ML models add. They point to the same leader per arm as the simple models, which is the strongest evidence that these are real. Their second and third places are softer than they look: `muscle_strength` scores 0.88 in Tideglusib yet LASSO kept it in only 6% of redraws; a tree can use a covariate for a few patients without a general effect existing.

8. The vote: signal, uncertain, noise

Each covariate gets one vote per method, five in total: stepwise selected it; LASSO kept it; elastic net kept it; random-forest permutation importance above the probe; XGBoost SHAP above the probe. Then:

- **Signal:** at least 4 votes and LASSO stability of 60% or more.
- **Uncertain:** at least 2 votes and stability of 20% or more. The stability floor matters because the two ML votes alone are a low bar.
- **Noise:** everything else. The probe itself scored 0 votes and 6 to 12% stability in every arm; that is the yardstick for "chance".

Arm	Signal (votes, stability)	Uncertain
Mexiletine	`myotonia_severity` (5, 100%); `ctg_repeats` (4, 70%)	`disease_duration` (3, 24%)
Oligonucleotide	`ctg_repeats` (5, 90%)	`myotonia_severity` (3, 43%); `disease_duration` (3, 30%); `muscle_strength` (2, 43%); `insulin_resistance` (2, 20%)
Tideglusib	`cognitive_function` (5, 100%)	`disease_duration` (4, 48%)

Section 5 shows up as predicted: in Mexiletine, stepwise kept `myotonia_severity` and dropped its twin `ctg_repeats` while elastic net kept both. In Oligonucleotide the pair appears again with `ctg_repeats` the stronger partner, and `myotonia_severity` "uncertain" is mostly an echo of it. `disease_duration` is the borderline case in every arm: stepwise keeps it, LASSO drops it. That is "maybe", and the slides should say so rather than promote it.

9. Three things that looked odd, and what they were

After the first run, three results needed explaining. Each was investigated on training patients only, by five-fold cross-validation, in `analysis/explorations/euibeom/`.

9a. Tideglusib: why did ML win?

The outcome does not fall gradually with `cognitive_function`; it steps. Mean score by eighths of `cognitive_function`, lowest to highest: 8.4, 7.6, 8.4, 6.1, 3.1, 3.3, 3.9, 3.7. A straight line fits a step badly; trees fit it easily.

Model (training set, five-fold R^2)	R^2
Straight line in `cognitive_function`	0.38
Smooth curve (spline, 3 df)	0.42
Step: below cut-off yes/no, cut-off re-chosen inside each fold	0.47
Step + `disease_duration`	0.49
Random forest, all covariates	0.45

Result: a linear model given the step matches the forest. This became the "Linear with cut-off" model in the main script. The cut-off, chosen by a one-level tree on training patients, is -0.48.

9b. Oligonucleotide: why is prediction so poor?

The `ctg_repeats` effect is real and steady: mean change by fifths of repeat length is -0.08, 0.40, 0.90, 1.43, 2.24. But patients with the same repeat length still differ a lot; the spread of the outcome drops only from 1.73 to 1.53 after accounting for repeats. Nothing tried adds reliably: log, spline, cut-off, each candidate second covariate, an interaction (five-fold R^2 0.17 to 0.25), and the random forest is worse (0.15). This is a weak-signal arm, not a missed pattern. A plain line in `ctg_repeats` is the right model, and the honest statement is that individual prediction is weak.

9c. Mexiletine: why did the responder model find almost nobody?

Two reasons. First, the label is a relative rule (myotonia fell by 45% or more of baseline), while the continuous outcome is the absolute fall, which mostly reflects room to improve: baseline `myotonia_severity` alone gives five-fold R^2 0.69 and nothing adds to it. Second, only 29% respond, so predicted probabilities rarely pass 0.5. Using the training responder share as the cut-off instead:

Cut-off (logistic model on `myotonia_severity` + `ctg_repeats`, five-fold)	**Recall**	**Precision**	**Accuracy**
0.5	0.34	0.50	0.71
Responder share (0.29)	0.71	0.51	0.72

The main script now uses the training responder share as the cut-off in every arm (0.29, 0.41, 0.48). It is fixed before the test set is seen.

10. Scoreboard on the locked test set

Every method is scored on the same held-out patients. Continuous outcome: RMSE and MAE (in the outcome's own units, so not comparable between arms) and R^2 (1 = perfect, 0 = no better than predicting the mean, negative = worse). Responder label: ROC-AUC (0.5 = coin flip), and recall, precision and accuracy at the responder-share cut-off. 95% intervals come from 1000 bootstrap redraws of the test patients.

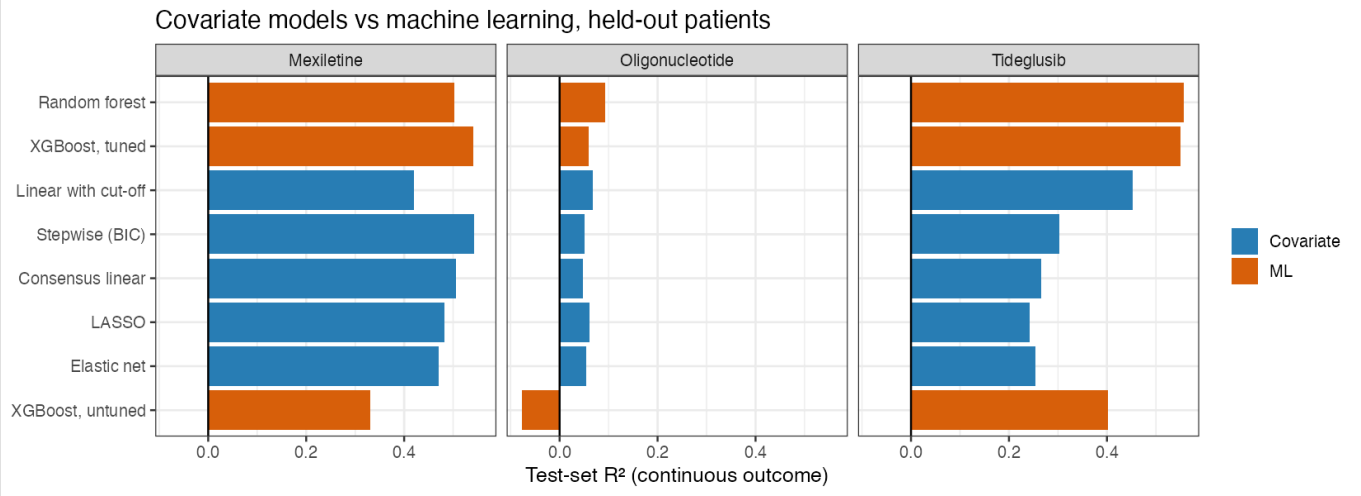


Figure 5. Test-set R^2 for every method, per arm. Blue = covariate models, orange = machine learning. Mexiletine: everything about 0.5. Oligonucleotide: everything near zero. Tideglusib: the two tree models at 0.55, the cut-off linear model at 0.45, the straight lines at 0.25 to 0.30. Untuned XGBoost is the worst or near-worst everywhere, which is what overfitting on a small arm looks like.

Arm	Method	RMSE	MAE	R^2	R^2 95% CI	AUC	Recall	Precision	Accuracy
Mexiletine test n = 41, 12 responders	Stepwise (BIC)	0.81	0.64	0.54	0.07 to 0.70	0.76	0.50	0.43	0.66
	Consensus linear	0.85	0.67	0.51	0.00 to 0.69	0.77	0.42	0.42	0.66
	LASSO	0.87	0.67	0.48	0.08 to 0.61	0.76	0.58	0.47	0.68
	Elastic net	0.87	0.68	0.47	0.09 to 0.60	0.77	0.75	0.50	0.71
	Linear with cut-off	0.92	0.70	0.42	-0.14 to 0.65	0.78	0.42	0.56	0.73
	XGBoost, tuned	0.82	0.67	0.54	0.10 to 0.71	0.77	0.83	0.48	0.68
	Random forest	0.85	0.65	0.50	0.10 to 0.67	0.73	0.67	0.44	0.66
	XGBoost, untuned	0.98	0.81	0.33	-0.41 to 0.62	0.60	0.25	0.43	0.68
Oligonucleotide test n = 43, 18 responders	Linear with cut-off	1.53	1.21	0.07	-0.33 to 0.29	0.64	0.56	0.59	0.65
	LASSO	1.53	1.17	0.06	-0.17 to 0.14	0.50	0.00	NA	0.58
	Elastic net	1.54	1.17	0.05	-0.17 to 0.13	0.72	0.67	0.60	0.67
	Stepwise (BIC)	1.54	1.19	0.05	-0.32 to 0.27	0.73	0.83	0.62	0.72
	Consensus linear	1.54	1.24	0.05	-0.32 to 0.25	0.72	0.67	0.60	0.67
	Random forest	1.51	1.19	0.09	-0.18 to 0.27	0.72	0.89	0.55	0.65
	XGBoost, tuned	1.53	1.22	0.06	-0.26 to 0.24	0.72	0.72	0.54	0.63
	XGBoost, untuned	1.64	1.29	-0.08	-0.72 to 0.29	0.73	0.78	0.58	0.67
Tideglusib test n = 41, 20 responders	Linear with cut-off	2.28	1.90	0.45	0.15 to 0.60	0.78	0.85	0.74	0.78
	Stepwise (BIC)	2.57	2.12	0.30	-0.04 to 0.50	0.79	0.85	0.74	0.78
	Consensus linear	2.64	2.19	0.27	-0.01 to 0.41	0.79	0.85	0.74	0.78

	Elastic net	2.66	2.21	0.25	0.05 to 0.36	0.79	0.85	0.74	0.78
	LASSO	2.68	2.22	0.24	0.05 to 0.34	0.79	0.85	0.74	0.78
	Random forest	2.05	1.72	0.56	0.37 to 0.66	0.87	0.85	0.74	0.78
	XGBoost, tuned	2.07	1.69	0.55	0.28 to 0.70	0.78	0.85	0.74	0.78
	XGBoost, untuned	2.39	1.92	0.40	-0.10 to 0.64	0.83	0.85	0.74	0.78

How to read this. The intervals are wide and overlap within every arm, so rankings between methods inside an arm are not meaningful; with about 41 test patients, one patient moves recall by 5 to 8 points. The one solid contrast is Tideglusib, straight lines against trees, and section 9a explains it. The Oligonucleotide LASSO row (AUC 0.50, recall 0) means the responder-label LASSO shrank every coefficient to zero and predicted the same probability for everyone: a null model, reported as it is. In Tideglusib all eight methods give identical recall and precision because every model ranks patients by `cognitive_function` and the cut-off lands in the same place.

11. Effect sizes and stratification

Direction and size of the consensus covariates

A linear model on the signal covariates, each standardised, so the coefficient is the change in outcome per one standard deviation of the covariate. Training patients; 95% confidence intervals.

Arm	Covariate	Change per 1 SD	95% CI	Reading
Mexiletine	`myotonia_severity`	-0.97	-1.24 to -0.70	One SD more myotonia at baseline, about one point more improvement (lower is better)
Mexiletine	`ctg_repeats`	-0.22	-0.48 to +0.05	Small and includes zero once myotonia is in the model: the twin adds little on its own
Oligonucleotide	`ctg_repeats`	+0.81	+0.54 to +1.08	One SD more repeats, 0.8 points more strength gain (higher is better)
Tideglusib	`cognitive_function`	-1.94	-2.37 to -1.51	One SD worse cognition at baseline, about two points higher composite score (higher is better)

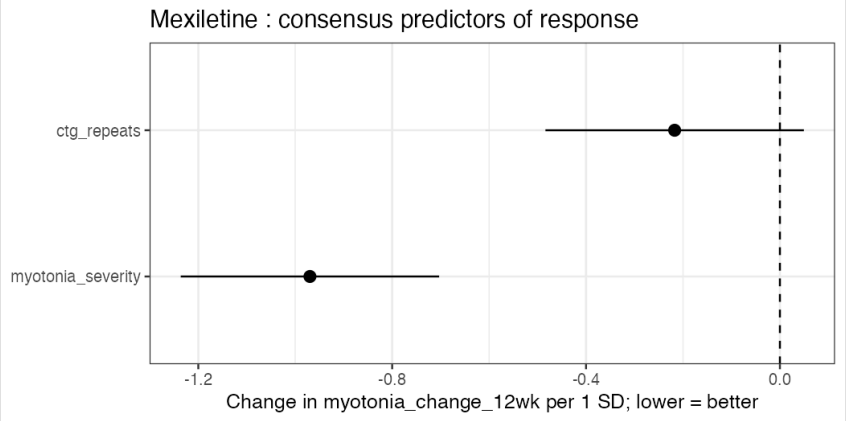


Figure 6. Mexiletine effect sizes. Dot = estimate, line = 95% CI, dashed line = no effect. `ctg_repeats` touches zero; `myotonia_severity` is far from it.

A rule that can be read aloud

A two-level decision tree is grown on training patients using only the signal covariates, so it cannot split on noise. Its leaves are then checked on the test patients. Responder rates below are among held-out patients.

Arm	Rule, chosen on training patients	Test n	Responders
Mexiletine	`myotonia_severity` below 4.85	20	10%
	`myotonia_severity` 4.85 or more, `ctg_repeats` below 458	15	40%
	`myotonia_severity` 4.85 or more, `ctg_repeats` 458 or more	6	67%
Oligonucleotide	`ctg_repeats` below 346	26	31%
	`ctg_repeats` 346 or more	17	59%
Tideglusib	`cognitive_function` -0.48 or more	18	17%
	`cognitive_function` below -0.48	23	74%

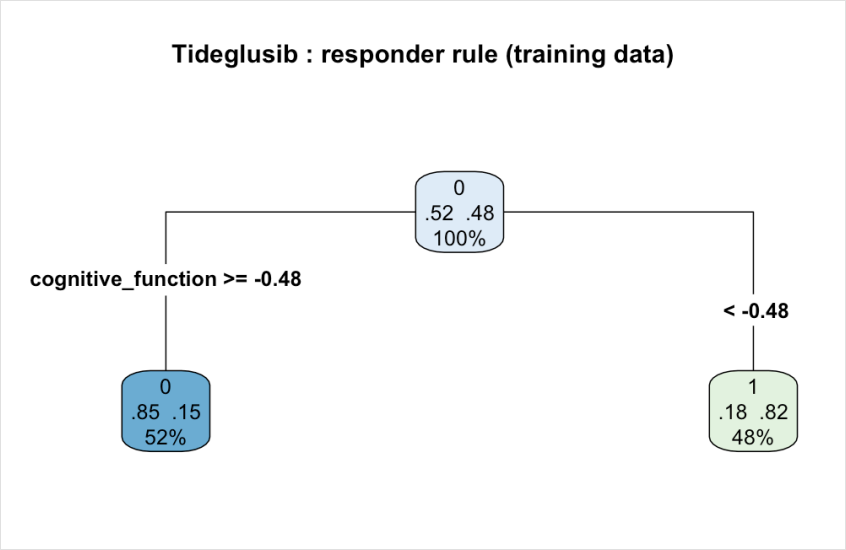


Figure 7. Tideglusib tree (training patients). Each box shows the majority class, the share of non-responders and responders, and the share of patients in that box. Right branch: cognition below -0.48, 82% responders in training, 74% in the test set.

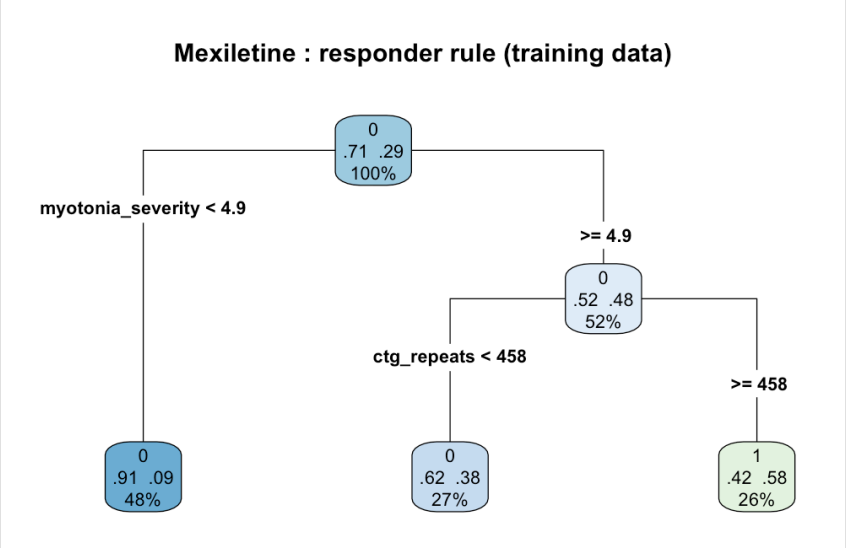


Figure 8. Mexiletine tree. The second split, on `ctg_repeats`, rests on 6 test patients. Present the first split as the rule and the second as a refinement.

Does predicted benefit sort real patients?

Test patients are ranked by the consensus linear model's prediction and cut into thirds. Observed responder rates: Mexiletine 7 / 31 / 50%; Oligonucleotide 20 / 50 / 57%; Tideglusib 21 / 46 / 79%. In every arm the "predicted high" third responds several times more often than the "predicted low" third.

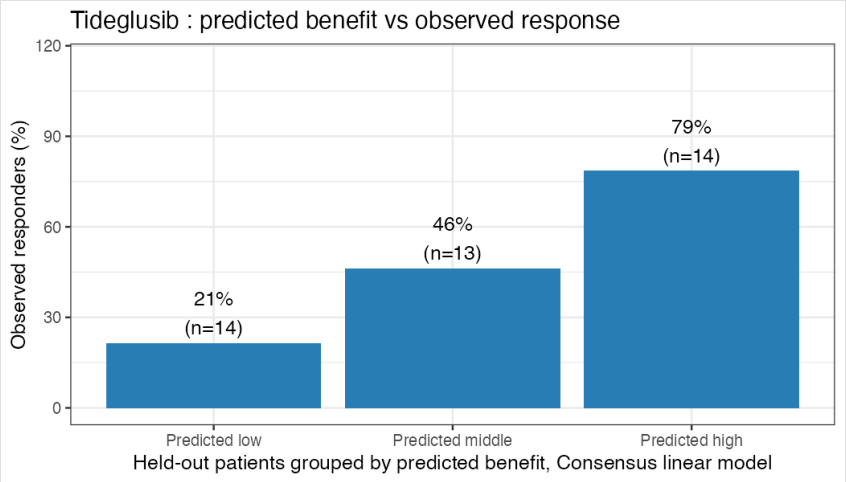


Figure 9. Tideglusib, held-out patients in thirds by predicted benefit.

12. Clinical reading, per arm

Arm	Mechanism	Who responds	Fit with mechanism	Stratification proposal
Mexiletine	Sodium-channel blocker; turns down over-excited muscle. Treats the symptom	Patients with more baseline myotonia (and, as a twin, longer repeats)	Direct: the drug acts on the symptom it is measured by. Patients with little myotonia have little to gain	Treat when myotonia severity is about 5 or more; expect little in mild myotonia
Oligonucleotide	Antisense strand that lowers the toxic *DMPK* RNA and frees MBNL1	Patients with longer CTG repeats	Direct: more repeats, more toxic RNA, more to remove. The organizers' hint was "molecular-level evidence might matter"	Prioritise repeat length above about 350; say that individual response remains hard to predict
Tideglusib	GSK3-beta inhibitor; multisystem, with a cognitive component in the composite outcome	Patients with impaired baseline cognition (z below about -0.5)	Plausible: the composite rewards cognitive gain, and the drug's target is linked to CNS effects. Patients with normal cognition have no cognitive room to improve	A single cut-off on cognitive function separates 74% from 17% responders

What was not found: `sex`, `bmi`, `smoking_status`, `cataracts`, `ck_level`, `pr_interval` and `albumin` were noise in every arm. The organizers planted noise covariates; these are the likely candidates, and none of them reached "uncertain" anywhere except `insulin_resistance` in Oligonucleotide (2 votes, 20%), which is borderline.

13. The code, block by block

Two files. `R/common.R` (112 lines) holds everything that must be identical for every teammate: loading, cleaning checks, the split, the folds, the metrics. `dm1_analysis.R` (about 240 lines) holds the analysis. Explorations source `common.R` and call `load_arm()` without the test set.

common.R

```
SEED <- 20261011
num_cov <- c("age", "age_at_onset", "disease_duration", "ctg_repeats", ...)   # 13 numeric
cat_cov <- c("sex", "phenotype", "family_history", "cataracts", "smoking_status") # 5
categorical
arms <- data.frame(name = c("Mexiletine", "Oligonucleotide", "Tideglusib"),
                   outcome = c("myotonia_change_12wk", "qmt_change_26wk",
"tideglusib_composite"),
                   label   = c("mexiletine_responder", ...), better = c("lower", "higher",
"higher"))
```

Fixed facts about the dataset: the seed, which columns are covariates, and for each arm the outcome column, label column and good direction.

```
load_arm <- function(name, include_test = FALSE) {
  d <- raw[!is.na(raw[[A$outcome]]), ]                 # this arm's patients
  d$.resp <- is_resp(d[[A$label]])                       # 0/1 responder
  set.seed(SEED + a)
  test_id <- unlist(lapply(split(seq_len(nrow(d)), interaction(d$.resp, d$phenotype, drop =
TRUE)),
                           function(i) sample(i, round(0.25 * length(i)))))
  tr <- d[-test_id, ]; te <- d[test_id, ]
  varies <- covs[sapply(tr[covs], function(x) length(unique(x)) > 1)]   # drops albumin in
Mexiletine
  out <- list(info = A, train = tr, covs = varies, dropped = setdiff(covs, varies))
  if (include_test) out$test <- te
  out }
```

The split. Patients are grouped by responder status and onset category, and a quarter of each group is drawn for the test set, which is what keeps both sides balanced. The test set is returned only when asked for, which only the main script does.

```
cv_folds <- function(train, k = 5) { set.seed(SEED)
  f <- integer(nrow(train))
  for (g in split(seq_len(nrow(train)), train$.resp)) f[g] <- sample(rep(seq_len(k), length.out
= length(g)))
  f }
reg_metrics <- function(y, p) c(RMSE = sqrt(mean((y - p)^2)), MAE = mean(abs(y - p)),
                                R2 = 1 - sum((y - p)^2) / sum((y - mean(y))^2))
cls_metrics <- function(y, p, cut = 0.5) { h <- as.integer(p >= cut)
  c(AUC = auc(y, p), Recall = sum(h & y) / sum(y), Precision = sum(h & y) / sum(h), Accuracy =
mean(h == y)) }
```

The five folds, balanced on responder status, and the metric formulas written out so there is nothing hidden.

dm1_analysis.R

```
fit_all <- function(tr, te, vars, y, family) {
  # Covariate models
  step <- MASS::stepAIC(update(full, . ~ 1), scope = ..., direction = "both", k =
log(nrow(dtr)))
  for (m in list(c("LASSO", 1), c("Elastic net", 0.5))) {
    cv <- cv.glmnet(Xtr, ytr, alpha = as.numeric(m[2]), family = family, nfolds = 5)
    pred[[m[1]]] <- predict(cv, Xte, s = "lambda.1se") }
  # Machine learning
  rf <- ranger(.y ~ ., data = dtr, num.trees = 500, importance = "permutation", seed = SEED)
  grid <- expand.grid(max_depth = c(1, 2, 3), eta = c(0.03, 0.1))
  ... xgb.cv(..., nfold = 5) for each grid row; keep the best; refit with that many rounds ...
  shap <- predict(x1, d1, predcontrib = TRUE) }
```

One function fits every model for one arm and one outcome type (`family = "gaussian"` for the continuous outcome, `"binomial"` for the label) and returns test-set predictions plus what each method selected. `k = log(n)` makes stepwise use BIC. `alpha` 1 is LASSO, 0.5 is elastic net; `lambda.1se` is the most-penalised setting within one standard error of the best, a standard conservative choice. `predcontrib = TRUE` gives SHAP values.

```
set.seed(SEED); tr$noise_probe <- rnorm(nrow(tr)); te$noise_probe <- rnorm(nrow(te))
vars <- c(covs_a, "noise_probe")
reg <- fit_all(tr, te, vars, A$outcome, "gaussian")
cls <- fit_all(tr, te, vars, ".resp", "binomial")
```

The noise probe is added, then everything is fitted twice: once per outcome type.

```
stab <- rowMeans(replicate(200, { i <- sample(nrow(tr), replace = TRUE)
  cv <- cv.glmnet(Xtr[i, ], tr[[A$outcome]][i], alpha = 1, nfolds = 5)
  by_var(as.numeric(coef(cv, s = "lambda.1se"))[-1], Xtr)[vars] > 0 }))
tab <- data.frame(covariate = vars,
  Stepwise = s$Stepwise > 0, LASSO = s$LASSO > 0, `Elastic net` = s$`Elastic net` > 0,
  `Random forest` = s$`Random forest` > s["noise_probe", "Random forest"],
  XGBoost = s$`XGBoost SHAP` > s["noise_probe", "XGBoost SHAP"], lasso_stability_pct =
round(100 * stab))
tab$votes <- rowSums(tab[2:6])
tab$call  <- ifelse(tab$votes >= 4 & tab$lasso_stability_pct >= 60, "signal",
             ifelse(tab$votes >= 2 & tab$lasso_stability_pct >= 20, "uncertain", "noise"))
```

Stability (200 redraws), the five votes, and the call rule from section 8.

```
keep <- setdiff(tab$covariate[tab$call == "signal"], "noise_probe")
cons <- lm(reformulate(keep, A$outcome), trs)                    # standardised covariates
stump <- rpart(reformulate(top, A$outcome), tr, control = rpart.control(maxdepth = 1, minbucket
= 15, cp = 0))
cut_at <- stump$splits[1, "index"]                               # e.g. -0.48 for
cognitive_function
reg$pred[["Linear with cut-off"]] <- predict(lm(reformulate(c(".below", keep[-1]), A$outcome),
trc), tec)
```

The consensus linear model (effect sizes in section 11) and the cut-off model from section 9a. The cut-off is the first split of a one-level tree on training patients.

```
cut_p <- mean(tr$.resp)                    # training responder share, e.g. 0.29
board <- ... reg_metrics(yt, reg$pred[[m]]); cls_metrics(rt, cls$pred[[m]], cut = cut_p) ...
R2_95CI = boot_ci(yt, reg$pred[[m]], function(y, p) reg_metrics(y, p)[["R2"]])
```

The scoreboard: the only place test outcomes (`yt`, `rt`) are compared with predictions. `boot_ci` redraws the test patients 1000 times for the intervals.

```
tree <- rpart(reformulate(keep, ".resp"), data = tr, method = "class",
              control = rpart.control(maxdepth = 2, minbucket = 15, cp = 0.02))
... leaf(te) ... responder rate per leaf among test patients ...
```

The read-aloud rule (section 11): grown on training patients from signal covariates only, with at least 15 patients per leaf, then checked on the test set.

14. Decisions, limits, AI use

Decision	Choice and reason
Primary outcome	Continuous, with the responder label alongside. The label is the continuous value cut at one point, so it carries less information; the label is used for the stratification message
Random-forest vote	Permutation importance above the noise probe. Boruta was used in the first run; replaced because permutation importance was taught at kickoff and gives the same leaders
Responder cut-off	Training responder share, not 0.5 (section 9c)
Cut-off linear model	Added after section 9a; keeps an explainable model competitive with ML
Class imbalance	No oversampling: not taught, risks leakage, irrelevant to the continuous outcome
Cleaning	Section 3; no patient removed

Limits to say aloud. About 41 test patients per arm, so intervals are wide and method rankings inside an arm are not meaningful. No placebo arm, so findings are "who does well on this drug", not "who benefits from it". Arm assignment was a weighted draw informed by patient fit, so arms are not random samples of one population. The Mexiletine two-step rule rests on 6 test patients. The test set has been scored twice; both runs are logged.

AI use. The scripts, this report and the team documents were drafted with Claude (Anthropic) through Claude Code. The code header and `AI_DISCLOSURE.md` say so. Covariate choices and interpretation are the team's to confirm.

To reproduce. `cd analysis && Rscript dm1_analysis.R`, about a minute; outputs land in `dm1_outputs/`. The three exploration scripts run with `Rscript` from `analysis/explorations/euibeom/`.